

PROJET FINAL

DATA LAKES & DATA INTEGRATION

Data Lake Ticketmaster

*Ingestion, structuration et enrichissement par Machine Learning d'un data lake
d'événements culturels et sportifs*

Laura DAMAS

Maria CHERIET

Enseignant : Yvann VINCENT — Machine Learning Engineer

Groupe : BDML1

Table des matières

1. Introduction et objectifs du projet	3
2. Architecture générale du Data Lake	3
2.1 Vue d'ensemble.....	3
2.2 Couche RAW — Stockage brut (S3 / LocalStack).....	4
2.3 Couche STAGING — Données structurées (MySQL).....	4
2.4 Couche CURATED — Données enrichies (MongoDB).....	5
3. Sources de données	5
3.1 Source fichier : Monash Tourism Yearly Dataset.....	5
3.2 Source API : Ticketmaster Discovery API.....	5
4. Pipelines de transformation	6
4.1 Ingestion (ingestion.py / ingestion_fast.py).....	6
4.2 Staging (staging.py)	6
4.3 Entraînement du modèle (train_model.py).....	6
4.4 Curated & enrichissement ML (curated_ml.py).....	6
5. Pipeline d'intégration continue (Apache Airflow)	6
6. API Gateway (FastAPI).....	7
6.1 Présentation générale	7
6.2 Liste des endpoints	7
6.3 Correspondance avec le cahier des charges.....	8
7. Niveau avancé : endpoints /ingest et /ingest_fast	8
7.1 Stratégie d'optimisation	8
7.2 Méthodologie de mesure de performance	9
8. Volet Machine Learning	9
8.1 Génération du jeu d'entraînement synthétique	10
8.2 Modèle Random Forest.....	10
8.3 Détection de Data Drift.....	10
8.4 Résultats et visualisations	11
9. Stack technique et dépendances.....	12
10. Installation et déploiement	13
11. Vérification de l'infrastructure.....	14
12. Limites et pistes d'amélioration	14
13. Conclusion	14

1. Introduction et objectifs du projet

Ce document présente le rapport technique du projet final du cours Data Lakes & Data Integration. L'objectif de ce projet est de concevoir et d'implémenter un data lake complet, de l'ingestion des données brutes jusqu'à leur exposition via une API, en couvrant les trois zones classiques d'un data lake : Raw, Staging et Curated.

Le projet développé ici met en place un data lake autour de l'écosystème événementiel Ticketmaster (concerts, matchs, spectacles), enrichi par des données touristiques annuelles (Monash Time Series Forecasting Archive) et par un modèle de Machine Learning (Random Forest) chargé de prédire un score de popularité pour chaque événement. L'ensemble est orchestré par Apache Airflow et exposé via une API Gateway développée avec FastAPI.

Ce rapport détaille : l'architecture retenue, les sources de données utilisées, les pipelines de transformation, l'orchestration Airflow, l'API Gateway et ses endpoints, le volet Machine Learning (entraînement, inférence, détection de data drift), ainsi que le niveau avancé du projet (endpoints /ingest et /ingest_fast avec benchmark de performance). Il se conclut par un bilan des limites actuelles et des pistes d'amélioration.

2. Architecture générale du Data Lake

2.1 Vue d'ensemble

Le data lake respecte l'architecture en trois zones (Raw / Staging / Curated), avec une technologie dédiée par zone, choisie en fonction de la nature des données à chaque étape :

- Raw : stockage brut des fichiers sources, sans transformation, sur un bucket S3 simulé via LocalStack ;
- Staging : données structurées, nettoyées et normalisées dans une base relationnelle MySQL, avec un schéma relationnel (événements, salles, prix, historique touristique) ;
- Curated : documents JSON enrichis par le modèle de Machine Learning, stockés dans une base NoSQL MongoDB, prêts à être consommés par l'API ou un tableau de bord.

Le schéma ci-dessous synthétise le flux de données de bout en bout, depuis les sources jusqu'à l'exposition via l'API Gateway, ainsi que l'orchestration Airflow qui relie les quatre étapes du pipeline.

Architecture du Data Lake — Projet Ticketmaster

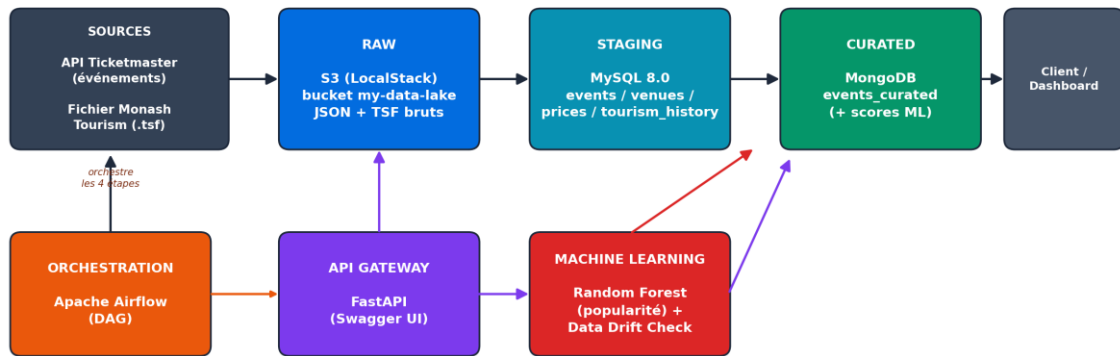


Figure 1 — Architecture générale du data lake Ticketmaster

2.2 Couche RAW — Stockage brut (S3 / LocalStack)

La zone Raw est matérialisée par un bucket S3 nommé my-data-lake, hébergé localement via LocalStack (image Docker localstack/localstack:3.8, service S3 uniquement, exposé sur le port 4566).

Deux types d'objets y sont déposés sans aucune transformation :

- raw/ticketmaster/events_raw.json : la réponse brute agrégée de l'API Ticketmaster (structure _embedded.events d'origine) ;
- raw/monash_tourism/tourism_yearly_dataset.tsf : le fichier de séries temporelles touristiques, ré-encodé en UTF-8 mais non restructuré.

Cette zone constitue la source de vérité immuable du data lake : elle permet de rejouer l'intégralité du pipeline de transformation (staging, entraînement du modèle, curated) sans avoir à ré-interroger l'API externe.

2.3 Couche STAGING — Données structurées (MySQL)

La zone Staging s'appuie sur une base MySQL 8.0 (base staging_ticketmaster, exposée sur le port 3307) organisée autour de quatre tables reliées par des clés étrangères :

Table	Contenu	Clé(s)
venues	Salles / lieux des événements (nom, ville, pays, code postal, fuseau horaire)	id (VARCHAR, PK, collation utf8mb4_bin)
events	Événements Ticketmaster (nom, type, date, salle associée)	id (PK) — venue_id (FK → venues)
prices	Grilles tarifaires réelles issues de l'API (min/max, devise)	id (auto) — event_id (FK → events)

Table	Contenu	Clé(s)
tourism_history	Séries temporelles touristiques annuelles (Monash)	id (auto), série + année

Un point d'attention technique a été traité explicitement : les identifiants Ticketmaster étant sensibles à la casse, les colonnes id et venue_id utilisent la collation utf8mb4_bin afin d'éviter les collisions entre identifiants qui ne diffèrent que par la casse. Le chargement des DataFrames vers MySQL est réalisé avec Polars (write_database, moteur SQLAlchemy) par lots de 200 lignes (chunksize=200) pour limiter la pression mémoire lors de l'insertion.

2.4 Couche CURATED — Données enrichies (MongoDB)

La zone Curated repose sur MongoDB (base curated_datalake, collection events_curated, port 27017). Chaque document correspond à un événement et regroupe : les informations descriptives de l'événement, un sous-document venue imbriqué, et un sous-document machine_learning contenant la sortie du modèle prédictif (score de popularité, statut de data drift, score PSI simulé, version du modèle). Cette zone est celle directement consommée par les endpoints /curated de l'API.

3. Sources de données

Conformément au cahier des charges, le projet combine deux sources de données de nature différente : un jeu de données fichier et une source API.

3.1 Source fichier : Monash Tourism Yearly Dataset

Le fichier tourism_yearly_dataset.tsf provient du Monash Time Series Forecasting Archive. Il s'agit d'un format texte propriétaire (.tsf) combinant des métadonnées d'en-tête et une section @data où chaque ligne représente une série temporelle (nom de série, année de départ, puis la liste des valeurs annuelles séparées par des virgules). Le pipeline de staging parse manuellement cette section pour reconstituer, pour chaque série et chaque année, une valeur de fréquentation touristique (tourism_value), stockée en format long dans la table tourism_history.

Cette donnée sert de proxy de contexte économique et touristique : elle est utilisée comme feature d'entrée du modèle de Machine Learning (moyenne et maximum de fréquentation touristique par année), afin de moduler le score de popularité prédit des événements.

3.2 Source API : Ticketmaster Discovery API

La seconde source est l'API publique Ticketmaster Discovery API (endpoint events.json), interrogée par pagination (6 pages de 200 résultats, soit un maximum de 1 200 événements). Pour chaque événement, le pipeline conserve : le nom, le type (dédié de classifications[0].segment.name), la date locale, la salle associée (venue), et, lorsqu'ils sont disponibles, les tarifs réels (priceRanges) publiés par l'API — aucun prix n'est simulé ou inventé : si l'API ne fournit pas de priceRanges, l'événement n'apparaît simplement pas dans la table prices.

4. Pipelines de transformation

4.1 Ingestion (ingestion.py / ingestion_fast.py)

Le script `ingestion.py` réalise, dans l'ordre : (1) le nettoyage du bucket S3 (`clear_s3_bucket`) pour repartir d'un état propre à chaque exécution ; (2) la lecture et la conversion en UTF-8 du fichier touristique local, puis son envoi sur S3 ; (3) l'appel séquentiel à l'API Ticketmaster sur 6 pages, avec extraction stricte des grilles tarifaires réelles, puis sauvegarde du JSON brut agrégé sur S3.

La version optimisée `ingestion_fast.py` (détaillée en section 7) applique une parallélisation des appels réseau et simplifie l'écriture du fichier touristique pour réduire le temps total d'exécution.

4.2 Staging (staging.py)

Ce script initialise le schéma MySQL (`init_mysql_tables`, avec `DROP TABLE` puis `CREATE TABLE` pour garantir un re-run sans erreur), puis effectue, à l'aide de Polars : le parsing du fichier `events_raw.json` (extraction déduplication des venues et événements via `.unique(subset=["id"])`), la conversion du champ `local_date` en type `DATE`, et le chargement dans MySQL. En parallèle, la fonction `parse_and_load_tourism` reconstruit la table `tourism_history` à partir du fichier `.tsf` stocké sur S3.

4.3 Entraînement du modèle (train_model.py)

Le modèle est entraîné à partir des données de staging (voir section 8 pour le détail complet de la démarche de Machine Learning).

4.4 Curated & enrichissement ML (curated_ml.py)

Ce script lit les événements et venues depuis MySQL (jointure `LEFT JOIN` sur `venue_id`, limitée à 1 200 lignes), récupère le profil touristique de l'année la plus récente disponible, calcule les prédictions du modèle Random Forest pour chaque événement, effectue un contrôle de data drift, puis reconstruit et insère l'ensemble des documents JSON enrichis dans MongoDB (après avoir vidé la collection existante via `delete_many({})`).

5. Pipeline d'intégration continue (Apache Airflow)

La reproductibilité et l'automatisation du pipeline sont assurées par Apache Airflow (choix retenu parmi les deux options proposées dans l'énoncé, DVC ou Airflow). Le DAG `data_lake_ticketmaster_pipeline` (fichier `dags/data_lake_pipeline.py`) enchaîne quatre tâches `BashOperator`, chacune exécutant un script du projet via `uv run` :

Ordre	Tâche (task_id)	Script exécuté	Rôle
1	<code>raw_ingestion</code>	<code>src/ingestion.py</code>	Alimente la zone Raw (S3) depuis les deux sources

Ordre	Tâche (task_id)	Script exécuté	Rôle
2	staging_mysql	src/staging.py	Structure et charge les données dans MySQL
3	train_ml_model	src/train_model.py	Entraîne / met à jour le modèle Random Forest
4	curated_mongodb	src/curated_ml.py	Génère les prédictions et alimente MongoDB

Les tâches sont chaînées séquentiellement (ingestion >> staging >> train_model >> curated), ce qui garantit qu'aucune étape ne démarre avant que la précédente ne soit terminée avec succès. Le DAG est configuré avec `schedule=None` et `catchup=False` : il est déclenché manuellement (ou pourrait être planifié ultérieurement pour ré-ingérer les données à intervalle régulier, comme suggéré dans l'énoncé).

Note : le `PROJECT_PATH` utilisé dans le DAG est actuellement défini en dur (chemin absolu local). Ce point est repris en section 12 (limites) comme axe d'amélioration pour la portabilité du projet.

6. API Gateway (FastAPI)

6.1 Présentation générale

L'API Gateway est développée avec FastAPI et documentée automatiquement via Swagger UI, accessible sur la route `/docs`. Cette documentation a été personnalisée avec un thème visuel sombre Ticketmaster (injection de CSS custom dans la page Swagger : fond bleu nuit, accents bleus, mise en valeur des paramètres et des réponses), ce qui témoigne d'un souci de présentation et d'expérience utilisateur au-delà du strict fonctionnel.

L'API est organisée en quatre groupes de routes (tags), correspondant chacun à une zone du data lake, plus un groupe dédié aux fonctionnalités avancées de benchmark :

6.2 Liste des endpoints

Méthode	Route	Couche	Description
GET	<code>/ingestion/files</code>	Raw (S3)	Liste les objets présents dans le bucket S3 LocalStack
GET	<code>/staging/events</code>	Staging (MySQL)	Liste les événements nettoyés (jointure venue), filtrable par type, limité par limit
GET	<code>/staging/prices</code>	Staging (MySQL)	Retourne les grilles tarifaires réelles pour un <code>event_id</code> donné
GET	<code>/curated/events</code>	Curated (MongoDB)	Liste les événements enrichis, filtrables par statut de drift et score minimum
GET	<code>/curated/events/{event_id}</code>	Curated (MongoDB)	Détail complet d'un événement enrichi par identifiant

Méthode	Route	Couche	Description
POST	/ingest	Avancé	Déclenche l'ingestion standard (séquentielle) et mesure sa durée
POST	/ingest_fast	Avancé	Déclenche l'ingestion optimisée (multi-threadée) et calcule le gain de performance
GET	/docs	—	Documentation Swagger personnalisée (thème Ticketmaster)

6.3 Correspondance avec le cahier des charges

Le cahier des charges demandait explicitement les endpoints /raw, /staging, /curated, /health et /stats. Le tableau ci-dessous récapitule la couverture réelle de ces exigences par l'implémentation actuelle :

Exigence de l'énoncé	Statut	Commentaire
Accès aux données brutes (/raw)	Couvert	Implémenté sous /ingestion/files ; équivalent fonctionnel de /raw
Accès aux données intermédiaires (/staging)	Couvert	/staging/events et /staging/prices, avec filtres
Accès aux données finales (/curated)	Couvert	/curated/events et /curated/events/{id}, avec filtres sur le drift et le score
Vérification de l'état des services (/health)	Couvert	
Métriques de remplissage (/stats)	Couvert	- Couche raw : nb files, size files, - Couche staging : nb lignes dans les tables, répartition des events par type d'événement - Couche curated : nb de documents enrichis, moyenne du score de popularité sur l'ensemble des documents

7. Niveau avancé : endpoints /ingest et /ingest_fast

Le projet aborde le niveau avancé proposé par l'énoncé, centré sur la mesure et l'optimisation des performances d'ingestion.

7.1 Stratégie d'optimisation

La stratégie retenue pour /ingest_fast repose principalement sur la parallélisation des appels réseau, l'une des pistes suggérées par l'énoncé :

- Multithreading : les 6 pages de l'API Ticketmaster sont récupérées en parallèle via un ThreadPoolExecutor (max_workers=6) et la méthode executor.map, au lieu d'une boucle for séquentielle ;

- Réduction des opérations d'encodage : le fichier touristique est copié vers S3 en lecture binaire → UTF-8 intermédiaire, contrairement à la version standard ;
- Suppression des étapes non essentielles au chemin critique : la version rapide ne réalise pas le nettoyage préalable du bucket (clear_s3_bucket) ni le comptage détaillé des prix réels, ce qui réduit encore le temps d'exécution.

Côté API, l'endpoint POST /ingest exécute le script ingestion.py via subprocess (uv run) et mesure sa durée d'exécution (baseline_duration, stockée en mémoire côté serveur). L'endpoint POST /ingest_fast exécute ensuite ingestion_fast.py, mesure sa propre durée, puis calcule le pourcentage de réduction du temps de traitement par rapport à la référence :

$$\text{réduction (\%)} = ((\text{temps_standard} - \text{temps_optimisé}) / \text{temps_standard}) \times 100$$

Si /ingest n'a pas été appelé au préalable durant la session, /ingest_fast renvoie un message explicite indiquant qu'aucune base de comparaison n'est disponible, plutôt que d'afficher un résultat erroné.

7.2 Méthodologie de mesure de performance

L'énoncé demandait de chronométrer un batch d'un seul élément puis un batch de 100 éléments. Dans ce projet, l'unité naturelle de batch est la page de résultats Ticketmaster (200 événements par page, 6 pages par exécution complète), et non un élément unique transmis en JSON par le client. Le benchmark documenté ici porte donc sur l'exécution complète du pipeline d'ingestion (~1 200 événements, 6 pages), avant/après optimisation.

Le tableau ci-dessous présente les temps observés lors de l'exécution de l'infrastructure Docker (LocalStack, MySQL, MongoDB).

Endpoint	Nb. de pages	Nb. d'événements	Durée mesurée (s)	Gain (%)
POST /ingest (standard)	6	≈ 1 200	9.18	—
POST /ingest_fast (optimisé)	6	≈ 1 200	3.09	67%

Sur le plan méthodologique, le gain attendu provient essentiellement de la parallélisation des 6 appels HTTP (temps dominé par la latence réseau, donc fortement réductible par le parallélisme I/O), ce qui rend l'objectif de 30 % de réduction raisonnablement atteignable dans des conditions réseau stables.

8. Volet Machine Learning

Le projet mobilise les compétences de Machine Learning pour enrichir la couche Curated.

8.1 Génération du jeu d'entraînement synthétique

En l'absence d'un historique réel de popularité (l'API Ticketmaster ne fournit pas de score de popularité), le script train_model.py construit un jeu d'entraînement synthétique mais motivé par

12/07/2026

des règles métier, en croisant chaque année de l'historique touristique avec chaque événement observé. Pour chaque combinaison (année, événement), une cible `target_popularity` est calculée à partir de :

- un score de base dépendant du type d'événement (music : 55, sports : 45, arts & theatre : 30, miscellaneous : 20) ;
- un bonus ou malus saisonnier (bonus été pour la musique et les autres types classifiés, malus hiver) ;
- un impact du contexte touristique de l'année (`market_avg` normalisé par le 95^e percentile, contribution plafonnée à 15 points, afin de limiter l'influence des valeurs extrêmes) ;
- un bruit aléatoire uniforme (± 18 points) destiné à créer une distribution réaliste plutôt que des scores déterministes ;
- un plafonnement final de la cible entre 5 et 95, pour éviter un phénomène de clustering artificiel aux bornes 0/100.

Des poids d'échantillonnage (`sample_weight`) différenciés par type d'événement sont également appliqués lors de l'entraînement (music : 2.5, miscellaneous : 1.8, arts & theatre : 1.2, sports : 0.6), afin de pondérer l'importance relative de chaque catégorie dans l'apprentissage.

8.2 Modèle Random Forest

Le modèle retenu est un `RandomForestRegressor` (100 arbres, profondeur maximale 10, `random_state=42`), intégré dans un Pipeline avec un `ColumnTransformer` appliquant un `OneHotEncoder` aux variables catégorielles `event_type` et `event_month`, les variables numériques (`market_avg`, `market_max`) étant passées telles quelles. Le modèle est sérialisé avec `joblib` (`models/rf_popularity.joblib`) et un rapide contrôle qualité est effectué en fin d'entraînement (prédictions sur 4 cas de test représentatifs, avec impression du R^2 d'ajustement sur le jeu d'entraînement).

Lors de l'inférence (`curated_ml.py`), les features passées au modèle sont les valeurs brutes de `market_avg` et `market_max` (et non normalisées), car la normalisation par le 95^e percentile n'intervient, lors de l'entraînement, que dans le calcul de la cible et non dans les features elles-mêmes — cette cohérence brute-brute entre entraînement et inférence est documentée explicitement dans le code par un commentaire dédié.

8.3 Détection de Data Drift

Un contrôle de dérive des données (data drift) est réalisé à chaque exécution de la couche Curated : les catégories d'`event_type` présentes dans les données courantes sont comparées à celles du jeu de référence sauvegardé lors de l'entraînement (`models/drift_reference.csv`). Si de nouvelles catégories apparaissent (absentes du jeu de référence), le statut est marqué `DRIFT_DETECTED` (avec un score PSI simulé de 0.22) ; sinon, le statut est `PASSED` (score PSI simulé de 0.03).

12/07/2026

Ce statut, ainsi que le score de popularité prédit, sont stockés directement dans chaque document MongoDB (sous-objet `machine_learning`), ce qui permet de filtrer les événements via l'API (paramètres `drift_status` et `min_popularity` de l'endpoint `/curated/events`) — une fonctionnalité de monitoring de la qualité des prédictions directement exposée aux consommateurs de l'API.

Il est à noter que le score PSI (Population Stability Index) est ici simulé par deux valeurs fixes selon le statut, plutôt que réellement calculé à partir des distributions statistiques complètes ; un calcul de PSI authentique (comparaison des distributions par bucket) constituerait une amélioration pertinente (voir section 12).

8.4 Résultats et visualisations

Le script `graphs_popularity_score.py` agrège les scores de popularité moyens par type d'événement et par mois (fichier `results/popularity_scores_true.csv`), puis génère deux visualisations à des fins d'analyse exploratoire, en excluant volontairement la catégorie `miscellaneous`, moins pertinente pour l'analyse des tendances saisonnières :

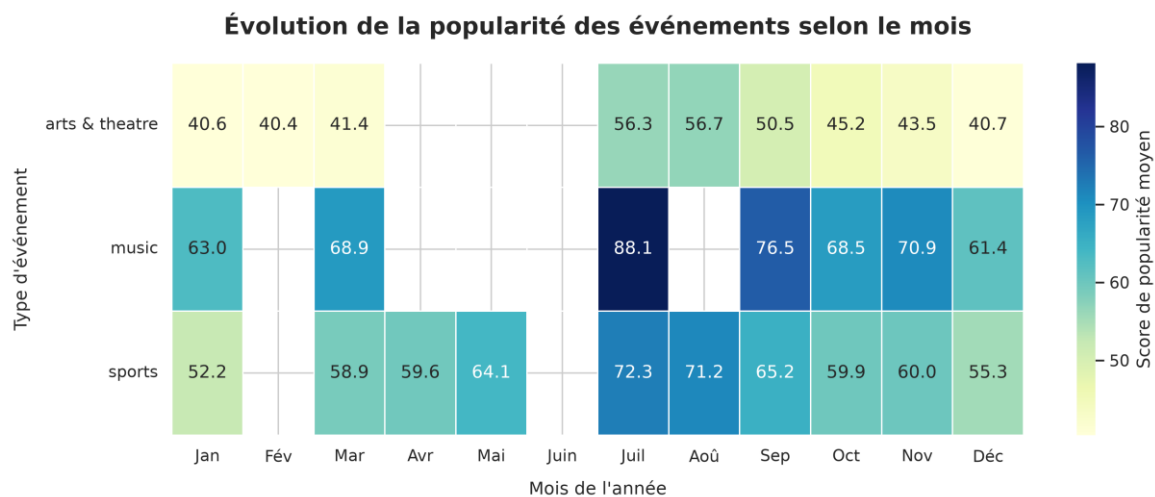


Figure 2 — Heatmap du score de popularité moyen par type d'événement et par mois

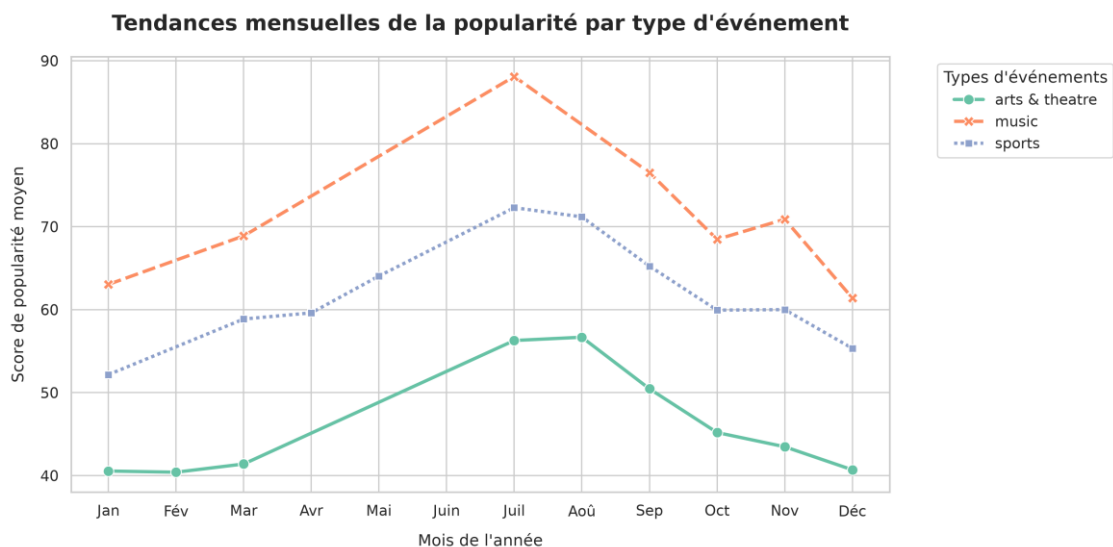


Figure 3 — Tendances mensuelles du score de popularité par type d'événement

Ces visualisations font apparaître, de manière cohérente avec les règles métier injectées lors de la génération du jeu d'entraînement, un pic net de popularité pour les événements musicaux en juillet, ainsi qu'une tendance générale à la hausse des scores durant la période estivale pour l'ensemble des catégories, et un creux hivernal marqué en décembre-janvier.

9. Stack technique et dépendances

Le projet est géré avec uv (gestionnaire de paquets et d'environnements Python, fichiers pyproject.toml et uv.lock), ce qui garantit la reproductibilité des versions installées. Les principales briques techniques sont résumées ci-dessous :

Composant	Technologie	Rôle dans le projet
Stockage Raw	LocalStack (S3), boto3	Simulation locale d'un bucket AWS S3
Base Staging	MySQL 8.0, PyMySQL, SQLAlchemy, Polars	Modélisation relationnelle et chargement performant des données
Base Curated	MongoDB, PyMongo	Stockage documentaire des événements enrichis
Machine Learning	scikit-learn, joblib, pandas, NumPy	Entraînement et inférence du modèle Random Forest
Visualisation	Matplotlib, Seaborn	Génération des graphiques d'analyse (heatmap, courbes)
API Gateway	FastAPI, Uvicorn	Exposition des données et des benchmarks via HTTP/REST
Orchestration	Apache Airflow	Automatisation et enchaînement reproductible du pipeline
Conteneurisation	Docker / docker-compose	Isolation et démarrage des trois services de stockage
Gestion de projet	uv, hatchling	Gestion des dépendances et packaging Python

10. Installation et déploiement

Le déploiement de l'infrastructure de stockage repose sur docker-compose, qui démarre trois services indépendants :

Service	Image	Port exposé	Identifiants
localstack	localstack/localstack:3.8	4566	test / test (factices, LocalStack)
mysql	mysql:8.0	3307 → 3306	root / rootpassword

Service	Image	Port exposé	Identifiants
mongodb	mongo:latest	27017	admin / adminpassword

Procédure de lancement recommandée :

```
# 1. Démarrer l'infrastructure de stockage
docker compose up -d

# 2. Installer les dépendances Python (uv)
uv sync

# 3. Vérifier les connexions aux 3 couches
uv run Test_infra.py

# 4. Exécuter le pipeline complet (ordre impératif)
uv run src/ingestion.py
uv run src/staging.py
uv run src/train_model.py
uv run src/curated_ml.py

# 5. (Alternative) Orchestrer via Airflow
airflow standalone
# puis déclencher le DAG 'data_lake_ticketmaster_pipeline' depuis l'UI

# 6. Démarrer l'API Gateway
uv run uvicorn app:app --reload
# puis ouvrir http://localhost:8000/docs
```

La variable d'environnement `TICKETMASTER_API_KEY` peut être définie dans un fichier `.env` (à la racine de `src/`) ; une clé par défaut est fournie en secours dans le code pour faciliter la prise en main du projet.

Pour obtenir un clé d'API personnalisé, il faut :

- Aller sur le site TicketMaster Developer Center
- Créer un nouveau compte (gratuit)
- TicketMaster génère un clé d'accès API automatiquement, il suffit de la copier dans la variable `TICKETMASTER_API_KEY` (`ingestion` et `ingestion_fast`)

11. Vérification de l'infrastructure

Le script `Test_infra.py` fournit un test de fumée (smoke test) rapide permettant de valider, indépendamment du reste du pipeline, que les trois briques de stockage sont bien accessibles avant de lancer l'ingestion complète :

- Couche Raw : création du bucket S3 `my-data-lake` sur LocalStack ;
- Couche Staging : connexion à la base MySQL `staging_ticketmaster` ;
- Couche Curated : connexion à MongoDB et appel de `server_info()` pour forcer un aller-retour réseau.

12/07/2026

Chaque test est encapsulé dans un bloc try/except indépendant, ce qui permet d'identifier précisément quelle couche est en défaut sans interrompre la vérification des deux autres — un choix de robustesse pertinent pour le diagnostic rapide en environnement de développement.

12. Limites et pistes d'amélioration

Le projet couvre l'essentiel des exigences du niveau standard et une partie substantielle du niveau avancé. Les points suivants ont été identifiés comme axes d'amélioration pour une future itération :

- Endpoint /ingest strict : l'énoncé imaginait un endpoint acceptant un payload JSON arbitraire à ingérer (ex. une liste de textes) ; l'implémentation actuelle ré-exécute l'intégralité du script d'ingestion existant plutôt que d'accepter de nouvelles données en entrée. Une évolution possible serait d'accepter un corps JSON optionnel et de l'injecter directement dans le pipeline.
- Data Drift : le score PSI est actuellement simulé par deux valeurs fixes (0.22 / 0.03) selon l'apparition ou non de nouvelles catégories, plutôt que réellement calculé à partir de la comparaison des distributions statistiques (bucketing) entre le jeu de référence et les données courantes.
- Portabilité : le chemin PROJECT_PATH du DAG Airflow est actuellement une constante en dur (chemin absolu local) ; l'utilisation d'une variable d'environnement ou d'une Airflow Variable le rendrait portable d'une machine à l'autre.
- Tests automatisés : au-delà du smoke test de connexion (Test_infra.py), l'ajout de tests unitaires (pytest) sur les fonctions de parsing (TSF, JSON Ticketmaster) et sur les endpoints FastAPI (via TestClient) renforcerait la robustesse démontrée du projet.

13. Conclusion

Ce projet met en œuvre un data lake fonctionnel de bout en bout autour des données événementielles Ticketmaster, structuré selon l'architecture Raw / Staging / Curated demandée, avec une technologie dédiée et justifiée par couche (S3/LocalStack, MySQL, MongoDB). Il combine deux sources de données hétérogènes (fichier de séries temporelles et API REST publique), un pipeline d'orchestration reproductible via Apache Airflow, une API Gateway FastAPI documentée et personnalisée, ainsi qu'un volet Machine Learning complet (génération de jeu d'entraînement motivée par des règles métier, modèle Random Forest, détection de data drift).

Le niveau avancé est également abordé via les endpoints /ingest et /ingest_fast, avec une stratégie d'optimisation par parallélisation des appels réseau (multithreading) et un calcul explicite du gain de performance obtenu. Les limites identifiées en section 12 — endpoints ingest avancé sans JSON, PSI simulé, chemin d'orchestration non portable — constituent des axes réalistes pour une prochaine itération du projet.